SKIT 1 — “Clone, Don’t Download”

Concepts: clone, status, add, commit, push, pull
Characters: Project Manager, Team Lead, Senior Developer, Developer, Intern, Tester

INT. OFFICE – MORNING

Project Manager:
“Today we are starting the sprint, and I want everyone to work from the same codebase. If people use different copies, we will waste time fixing simple mistakes.”

Team Lead:
“That is why everyone should clone the repository from GitHub instead of downloading a ZIP file. When you clone, your local code stays connected to the remote repository.”

Intern:
“I have cloned the project, and it looks like a normal folder on my laptop. So how is this better than just copying the files from somewhere?”

Senior Developer:
“The difference is that Git now knows where this code came from and can track every change you make. Because of that, you can pull updates and push your own work safely.”

Developer:
“I have checked git status, added my changes, and committed them with a proper message. Now I am pushing them so the rest of the team can see what I did.”

Tester:
“I just pulled the latest changes from the remote repository, and the build is still working fine. This makes testing easier because I always test the latest code.”

Project Manager:
"This is exactly what I want to see, because it proves that everyone is staying in sync. When we work like this, we avoid many unnecessary problems."

Intern:
"So now I understand that cloning is not just copying files, but joining the team workflow. I will also commit and push my changes instead of keeping them only on my laptop."

Team Lead:
"Good, and if everyone follows this daily process, our collaboration will be much smoother."

SKIT 2 — "Fork First, Then Pull Request"

Concepts: fork, clone fork, upstream, pull request
Characters: Client, Project Manager, Team Lead, Senior Developer, Developer, Intern

INT. MEETING ROOM – DAY

Client:
"We want your team to contribute to our open-source project, but we cannot give you direct access to our main repository. Our main branch is used by many users, so it must stay protected."

Project Manager:
"That sounds reasonable, because a small mistake could affect many people. So we need a way to work on your code without touching your main project directly."

Team Lead:
"In this case, we should fork your repository to our own GitHub account.

Forking creates our own copy, so we can experiment safely without risking your original code.”

Intern:
“I have a question about that, because after forking, do we still need to clone the project to our laptops? Or can we work only on GitHub?”

Senior Developer:
“We still need to clone our fork, because real development happens locally on our systems. The fork is just our own remote copy on GitHub.”

Developer:
“That also means we can commit and push our changes to our fork without any danger to the client’s main repository.”

Project Manager:
“But what if the client updates their project while we are still working on our copy? Won’t our code become outdated?”

Senior Developer:
“That is why we add the client’s repository as upstream, so we can pull their latest changes and stay in sync with their work.”

Intern:
“So when our feature is ready, we should not merge it ourselves, but ask the client to review it, right?”

Team Lead:
“Exactly, we will open a pull request and explain our changes clearly, so the client can review and decide whether to merge them.”

Client:
“This workflow keeps our project safe and also helps us review your contribution properly. Please follow this same process for all future changes.”

SKIT 3 — “Why We Use Branches”

Concepts: branch, checkout, pull request, merge, review
Characters: Project Manager, Team Lead, Senior Developer, Developer, Intern, Tester

INT. OFFICE – DAY

Project Manager:
“This sprint has three features, and I do not want the main branch to become unstable. If main breaks, the whole team will be blocked.”

Team Lead:
“That is exactly why we should not work directly on main. Everyone should create a feature branch before starting any new task.”

Intern:
“I am still learning, and I am worried that I might break something important on main. Does a branch really keep my mistakes separate from others?”

Senior Developer:
“Yes, that is the purpose of a branch, because it lets you work safely without affecting the stable code. When your work is ready, we review it before merging.”

Developer:
“I have already created my feature branch and checked it out. I will work there and open a pull request instead of merging directly.”

Tester:
“That helps me as well, because I only test features after they are merged and considered stable. Testing half-finished code usually wastes time.”

Project Manager:
“So the idea is that main stays clean, and features come in only after

review and testing. This should reduce last-minute surprises before release."

Intern:
"That makes me more comfortable, because I can experiment in my branch and fix my mistakes before anyone else sees them."

Team Lead:
"Exactly, and when the pull request is reviewed and approved, we can merge it confidently into main."

Senior Developer:
"This workflow also makes it clear who worked on what, which helps a lot during reviews and bug fixes."

SKIT 4 — "The Merge Conflict"

Concepts: pull, merge conflict, resolve conflict, commit
Characters: Project Manager, Team Lead, Senior Developer, Developer, Intern, Tester

INT. OFFICE – AFTERNOON

Project Manager:
"The build is failing, and the team cannot move forward because of it. Can someone explain what exactly caused this problem?"

Team Lead:
"It looks like two people changed the same file in different ways, and now Git is showing a merge conflict. Git cannot decide which change is correct."

Developer:
"I pulled the latest code, and now I see strange markers in the file showing two versions of the same lines. It looks like Git is asking me to choose."

Intern:
"I think this happened because I did not pull the latest changes before I started working. I was still working on an old version of the file."

Senior Developer:
"That is a common situation, and Git cannot resolve it automatically in this case. We need to open the file, understand both changes, and keep the correct parts."

Tester:
"Once you resolve it and commit the fixed version, I will test again to make sure the build is stable."

Project Manager:
"So pulling before starting work can reduce these conflicts, but sometimes they still happen when two people touch the same code."

Developer:
"Yes, and when they do happen, we should not panic, because Git is just asking us to make a clear decision."

Team Lead:
"After the conflict is resolved and committed, we should push the fix so everyone gets the clean version."

Senior Developer:
"And this is also why communication matters, because if we know who is working on what, we can avoid many conflicts."

SKIT 5 — "Rebase, Revert, and the Release"

Concepts: rebase, revert, tag
Characters: Client, Project Manager, Team Lead, Senior Developer, Developer, Tester

INT. MEETING ROOM – PRE-RELEASE

Client:
"We want this release to be stable, and we also want the commit history to look clean and easy to understand."

Project Manager:
"Our feature branch has many small commits, and the history looks messy right now. I am worried this will confuse anyone who looks at it later."

Team Lead:
"In that case, we can rebase the feature branch on main to make the history look cleaner before we merge it."

Senior Developer:
"We should remember that rebase rewrites history, so it must be done only on personal branches and not on shared branches."

Developer:
"I will rebase carefully and inform everyone before pushing the changes, so nobody gets confused or loses work."

Tester:
"While checking the latest build, I noticed that the last commit broke the login feature, so we should undo that change safely."

Team Lead:
"For that situation, we should use git revert, because it creates a new commit and keeps the history safe instead of rewriting it."

Project Manager:
"Once everything is stable and tested, we should also create a tag for this release so we can easily return to this version later."

Client:
"That sounds good, because we want both a clean history and a reliable release for our users."

SKIT 6 — “Stash Before the Demo”

Concepts: git stash, git stash pop
Characters: Project Manager, Team Lead, Senior Developer, Developer, Intern, Tester

INT. OFFICE – DAY

Project Manager:
“The client is coming for a demo in one hour, and they only want to see stable features, not half-finished work.”

Developer:
“My current feature is not ready, and I cannot commit it yet, but I also need to switch branches for the demo.”

Senior Developer:
“This is a good time to use git stash, because it can save your unfinished changes without committing them.”

Intern:
“So stash will give me a clean working directory, and I can bring my changes back later, right?”

Team Lead:
“Yes, after the demo you can use git stash pop and continue your work from where you stopped.”

Tester:
“The demo build looks stable now, and I do not see any unexpected behavior.”

Developer:
“I have applied my stashed changes back, and now I can continue working on my feature.”

Project Manager:
"This is a good example of how stash helps us manage unfinished work without disturbing important demos."

SKIT 7 — "Cherry-Pick the Hotfix"

Concepts: git cherry-pick, hotfix workflow
Characters: Project Manager, Team Lead, Senior Developer, Developer, Intern, Tester

INT. OFFICE – EMERGENCY MODE

Project Manager:
"We have a serious issue in production, and users are already reporting failures. We need a fix quickly, but we also need to be careful."

Team Lead:
"Our main branch must stay stable, and the feature branch still has unfinished work. Merging everything right now could create more problems."

Developer:
"I checked the commit history, and I can see one specific commit that fixes only this bug. That commit does not include any risky changes."

Intern:
"So instead of merging the whole branch, we can take only that one commit and apply it to main, right? That sounds much safer to me."

Senior Developer:
"Yes, and this is exactly when we use git cherry-pick, because it allows us to move a single commit from one branch to another. This way, we fix the bug without bringing unfinished features."

Tester:
"If you apply only that fix, I can test it quickly and confirm whether the issue is really solved."

Developer:
"I have cherry-picked the commit to the main branch and pushed the hotfix. The change is small and focused only on the reported problem."

Tester:
"I tested the hotfix in the staging environment, and the original issue is gone without any side effects."

Project Manager:
"Good, this approach saved us time and also reduced risk. Let's remember this pattern for future emergency fixes."

Team Lead:
"Yes, cherry-pick is very useful when we want a precise fix instead of a big merge."

SKIT 8 — "Recovering a Deleted Branch"

Concepts: git reflog, restore branch
Characters: Project Manager, Team Lead, Senior Developer, Developer, Intern, Tester

INT. OFFICE – QUIET PANIC

Developer:
"My feature branch is missing, and I think I deleted it by mistake while cleaning old branches. I am worried because that branch had two days of work."

Project Manager:
"That work is important for this sprint, so we cannot afford to lose it. Let us first understand whether Git can help us recover it."

Intern:
"I was trying to remove unused branches, and I might have deleted the wrong one. I did not realize it had active work on it."

Senior Developer:
"Do not panic, because Git keeps a history of recent actions in something called reflog. We can use git reflog to find the commit where that branch was pointing."

Team Lead:
"If we get that commit hash, we can create the branch again and continue from where we stopped. This is much better than rewriting everything."

Developer:
"I can see my commit in the reflog now, so it looks like the work is not really lost. I will recreate the branch from that commit."

Tester:
"Once the branch is back, I will test the features to make sure nothing was corrupted during recovery."

Project Manager:
"This is a good lesson, because Git history can save us, but we should still be careful when deleting branches."

Intern:
"I will definitely double-check before deleting anything next time."

SKIT 9 — "Fetch or Pull?"

Concepts: git fetch, git pull, when to use each
Characters: Project Manager, Team Lead, Senior Developer, Developer, Intern, Tester

INT. OFFICE – DESKS

Project Manager:
"I noticed that you are working on an older version of the code, but the remote repository already has some new updates. That can cause problems later."

Developer:
"I did not pull the changes because I was worried that a direct merge might break my current work. I wanted to be careful before changing anything."

Senior Developer:
"In this situation, you should use git fetch first, because it only downloads the latest changes without touching your working branch. That way, you can review what changed safely."

Intern:
"So fetch is only to see what is new, and pull is what actually merges those changes into my branch, right? That makes the difference much clearer."

Team Lead:
"Yes, and after fetching, you can compare and decide whether it is safe to pull. This gives you more control over what gets merged."

Developer:
"I have fetched the changes and reviewed them, and they look compatible with my work. Now I feel confident to pull them into my branch."

Tester:
"That is good, because I always want to test the latest integrated code, not something that is already outdated."

Project Manager:
"So using fetch first helps us avoid surprises, and pull is used when we are ready to merge. Let us follow this approach more often."

Senior Developer:
"Exactly, choosing the right command at the right time keeps the workflow clean and predictable."

SKIT 10 — "The Pull Request Blocked by CI"

Concepts: CI checks, PR review, fix and update
Characters: Project Manager, Team Lead, Senior Developer, Developer, Intern, Tester

INT. OFFICE – PROJECTOR ON

Project Manager:
"We are close to the release date, so I want to understand why this pull request is still not merged into main. Is there any serious issue?"

Team Lead:
"The CI checks failed, which means the automated tests found some problems in the code. Until those checks pass, we should not merge anything."

Developer:
"It worked fine on my local machine, but clearly the automated tests are catching something I missed. I need to look at the test reports carefully."

Intern:
"So CI is like an automatic reviewer that checks every change before it is allowed into main, right? That sounds strict but useful."

Senior Developer:
"Yes, and that is exactly why we trust it, because it checks things

consistently every time. If CI fails, we should fix the issues instead of ignoring them.”

Tester:
“I can also run some manual tests once the fixes are pushed, so we are confident from both automation and human testing.”

Developer:
“I have fixed the issues reported by CI and pushed the updated changes. The checks are running again now.”

Team Lead:
“The CI checks are green now, so technically the code is safe to merge. This shows why automated testing is important.”

Project Manager:
“Great, now we can merge this pull request and move forward with the release. This is a good example of a professional workflow.”

Intern:
“I understand now that CI is not blocking us, but actually protecting the project.”

Classroom Instructions for Git Roleplay + Live Demo (Using Projector)

🎯 Objective

Students will:

- Act out the **skits** (roles like PM, Dev, Tester, etc.)
- **Run real Git commands live** on the projector
- Show the **actual workflow**: clone, branch, commit, PR, merge, conflict, stash, etc.
- Learn **industry-style discipline**: backup folder vs presentation folder

---

🗂️ Folder Structure Rule (Very Important)

Each team must create **TWO folders**:

📁 TeamX_Project_MAIN ← Used for presentation (clean, stable)
📁 TeamX_Project_BACKUP ← Used for experiments, practice, risky work

🔹 How they should work:

- ✅ Do **real work & practice** in: `TeamX_Project_BACKUP`
- ✅ Keep **stable, demo-ready code** in: `TeamX_Project_MAIN`
- ✅ During presentation:
  - Open ONLY `TeamX_Project_MAIN`
  - Show Git history, branches, PRs, tags, etc.
- ❌ Do NOT break the MAIN folder before demo

This mimics **real industry**:

- `MAIN = production/stable`
- `BACKUP = development/experiments`

🖥️ Classroom Setup (Projector Workflow)

Each team (or selected team per class):

1. Connect laptop to **projector**
2. Open:
    - Terminal / Git Bash
    - VS Code (or any editor)
    - GitHub repo in browser
3. One student **acts + speaks dialogue**
4. Another student **executes commands live**
5. Others **explain what is happening**

How to Perform Each Skit (Step-by-Step)

Before Starting

- Create GitHub repo (or fork, depending on skit)
- Clone into both folders:

```
 git clone <repo-url> TeamX_Project_MAIN
git clone <repo-url> TeamX_Project_BACKUP
```

During Skit Performance

Students must:

- 🔹 Speak the **dialogue**
- 🔹 At the same time, **execute commands** like:

Skit 1 (Clone, Status, Commit, Push, Pull):

```
git status
git add .
git commit -m "Add login page"
git push
git pull
```

Skit 2 (Fork, Upstream, PR):

```
git remote -v
git remote add upstream <original-repo>
git fetch upstream
git pull upstream main
```

(Then show PR on GitHub)

Skit 3 (Branch Workflow):

```
git branch feature-login
git checkout feature-login
git push -u origin feature-login
```

Skit 4 (Merge Conflict):

```
git pull
# Show conflict markers in file
# Fix file
git add .
git commit -m "Resolve merge conflict"
```

Skit 5 (Rebase, Revert, Tag):

```
git rebase main
git revert <commit-hash>
git tag v1.0
git push origin v1.0
```

Skit 6 (Stash):

```
git stash
git checkout main
git stash pop
```

Skit 7 (Cherry-pick):

```
git cherry-pick <commit-hash>
```

Skit 8 (Reflog & Restore):

```
git reflog
git checkout -b recovered-branch <commit-hash>
```

Skit 9 (Fetch vs Pull):

```
git fetch
git log origin/main
git pull
```

Skit 10 (CI + PR):

- Show:
    - GitHub PR
    - ❌ Failed checks
    - ✅ Fixed commit
    - ✅ Green checks
    - Merge PR

How to Structure (Quick Plan)

- **60 students → 10 teams × 6 members**
- Each team gets:
    - 1 Product
    - Roles:
        - Team Lead / PM
        - Backend Dev
        - Frontend Dev
        - Database / API Dev

        - QA / Testing
        - DevOps / Git Manager
- Workflow:
    - GitHub Issues = Tasks
    - Branch per feature
    - Pull Requests + Reviews
    - Weekly “Sprint Update” in README / Project Board

💡 12 Real-Time Industry-Level Project Ideas

🏦 1. Smart Expense & Budget Control System (FinTech)

Like: Splitwise + Wallet + Reports
Features:

- User accounts, monthly budgets
- Expense categories, charts
- Alerts when budget exceeds
- Export reports (PDF/CSV)
- Admin dashboard

Modules to split:

- Auth, Expense API, Reports, UI, Charts, Alerts, GitOps

🧾 2. Mini Digital Banking System

Like: NetBanking demo
Features:

- Account creation, balance, transactions
- Transfer between users
- Transaction history
- Admin panel to block/unblock users
- Basic fraud flag (rule-based)

🏢 3. Employee Management + Payroll System

Like: Zoho People / BambooHR (mini)
Features:

- Employee CRUD
- Attendance tracking
- Leave requests & approvals
- Salary calculation

- Payslip generation

📊 4. Project Management Tool (Mini Jira / Trello)

Meta project 😄
Features:

- Projects, tasks, sprints
- Assign tasks to users
- Status: ToDo / InProgress / Done
- GitHub integration (link commits)
- Activity log

🧠 5. AI-Assisted Resume Screening System

Features:

- HR uploads job description
- Candidates upload resumes
- System scores resumes (keyword match)
- Shortlist dashboard
- Export shortlist

🛒 6. B2B Inventory & Billing System

Like: Small business ERP
Features:

- Products, stock, suppliers
- Purchase & sales invoices
- GST/Tax calculation
- Low-stock alerts
- Sales reports

🏫 7. Campus Placement Management Portal

Features:

- Student profiles
- Company postings
- Apply for jobs
- Shortlisting & status tracking
- Admin & TPO dashboard

🏥 8. Hospital Appointment & Records System

Features:

- Patient registration
- Doctor schedules
- Appointment booking
- Visit history
- Admin analytics dashboard

🏦 9. Loan Management System (FinTech Core)

Features:

- Apply for loan
- EMI calculator
- Approval workflow
- Payment tracking
- Defaulter reports

📈 10. Business Analytics Dashboard Platform

Like: PowerBI-lite (web)
Features:

- Upload CSV data
- Generate charts
- Filter & search
- Save dashboards
- Share reports

👩‍💼 11. Freelance Marketplace System

Like: Fiverr / Upwork mini
Features:

- Client & freelancer accounts
- Post jobs
- Bid system
- Work status tracking
- Payment status (mock)

🛡️ 12. Complaint / Case Management System (GovTech)

Features:

- Citizen submits complaint
- Assign to officer
- Status tracking
- Evidence upload
- Reports & analytics

🧩 How Each Team Can Split Work (Example)

For each project:

- 🔹 Frontend (UI, forms, dashboards)
- 🔹 Backend APIs (Auth, CRUD, business logic)
- 🔹 Database design
- 🔹 Reports / Analytics module
- 🔹 Admin panel
- 🔹 GitHub + CI + Documentation

Each student owns **one module** → pushes via **PR** → reviewed → merged.

📌 GitHub Workflow You Can Enforce

- One **org/repo per team**
- Use:
  - Issues = Tasks
  - Projects board = Status tracking
  - Branch naming: `feature/login`, `feature/reports`
  - PR template + Code review
- Weekly:
  - Each team updates `STATUS.md`
  - Demo + GitHub activity check

How to Track Work & Assign to Team Members

🔹 Step 1: Add Your Team Members to the Repository

1. Open your GitHub repository in browser
2. Click **Settings** (top menu of repo)

3. Click **Collaborators** (or **Collaborators & Teams**)
4. Click **Add people**
5. Enter your friend's **GitHub username / email**
6. Click **Invite**
7. They must **accept the invitation**

✅ Now they can:

- Push code
- Create branches
- Open issues
- Work on tasks

🔹 Step 2: Decide Your Work Items (Tasks)

Before creating anything, list tasks like:

- Setup project structure
- Design login UI
- Create database schema
- Implement authentication

- Build dashboard
- Write API for orders
- Testing & bug fixes
- Documentation

Each of these = **ONE work item (task)**

🔹 Step 3: Create Issues (This is Your Task Tracker)

1. Go to your GitHub repo
2. Click **Issues** tab
3. Click **New issue**
4. Add:
    - **Title** → e.g. `Create Login Page UI`
    - **Description** → What needs to be done, brief steps, expectations
5. On the right side:
    - **Assignees** → Select the teammate responsible

- **Labels** → e.g. `frontend`, `backend`, `bug`, `enhancement`
- (Optional) **Milestone** → e.g. `Sprint 1`

6. Click **Submit new issue**

✅ Repeat this for **each task**

Now you have:

- One issue = one work item
- Each issue = assigned to one student

🔹 Step 4: Use GitHub Projects (Kanban Board Style) (Optional but Pro-Level)

1. In your repo, click **Projects** (or **Projects** from GitHub top menu)
2. Click **New project**
3. Choose **Board** (Kanban style)
4. Name it: `TeamX Sprint Board`
5. Create columns:
   - 📝 Todo
   - 🚧 In Progress

- ✅ Done

6. Add your **Issues** to this board

7. Move cards as work progresses:

   - Todo → In Progress → Done

✅ Now you have a **visual task tracker** like Jira/Trello

🔹 Step 5: How Each Student Should Work on Their Task

For each issue:

1. Student opens the **Issue assigned to them**

2. Creates a **new branch**:

   ```
   feature-login-ui
   fix-order-bug
   add-dashboard-api
   ```

3. Works only on that branch

4. Commits changes:

   ```
   git add .
   git commit -m "Add login UI"
   git push origin feature-login-ui
   ```

5. Opens a **Pull Request (PR)**:
    - From: `feature-login-ui`
    - To: `main`
6. In PR description, write:

    Fixes #3

    (This links the PR to Issue #3)

🔹 Step 6: Review, Merge, and Close Task

1. Teammates / Lead reviews the PR
2. If OK → Click **Merge**
3. GitHub will **automatically close the issue** if you wrote `Fixes #issueNumber`
4. Move card in Project Board to:
    ✅ Done

🔹 Step 7: How Project Manager / Team Lead Tracks Progress

They should:

- Open **Issues tab** → See who is working on what
- Open **Projects board** → See:
  - What is pending
  - What is in progress
  - What is completed
- Open **Pull Requests** → Review code before merging

🏢 This Matches Real Industry Workflow

- ✅ Issues = Work items / tickets
- ✅ Assignees = Responsibility
- ✅ Branch per task = Safe development
- ✅ Pull Request = Code review
- ✅ Projects board = Sprint tracking
- ✅ main branch = Stable code

👩‍🏫 For Your Classroom Demo (Projector)

Ask students to **show live**:

- Creating an Issue
- Assigning it to a friend
- Creating a branch for that issue
- Pushing code
- Opening PR
- Merging PR
- Issue moving to **Done** ✅